

Bluetooth Fundamentals

Bluetooth Fundamentals

1. Bluetooth Overview
 - 1.1 What Is Bluetooth
 - 1.2 Bluetooth Version Evolution
 - 1.3 Bluetooth Band and Channels
 - 1.4 Bluetooth vs WiFi
2. Bluetooth Protocol Stack Architecture
 - 2.1 Dual-Mode Architecture
 - 2.2 Host and Controller Layering
 - 2.3 Detailed Layer Descriptions
3. Classic Bluetooth vs Bluetooth Low Energy
 - 3.1 Core Differences
 - 3.2 Why BLE Has Low Power Consumption
4. BLE Core Concepts
 - 4.1 GAP — Generic Access Profile
 - The Four GAP Device Roles
 - Advertising Types
 - Advertising Data Format
 - 4.2 ATT — Attribute Protocol
 - Attribute Structure
 - GATT Operations and Permission Mapping
 - 4.3 GATT — Generic Attribute Profile
 - UUID Structure
 - CCCD — Client Characteristic Configuration Descriptor
 - 4.4 Notify vs Indicate Comparison
 - 4.5 BLE Connection Parameters
5. BLE Advertising and Connection Process
 - 5.1 Complete Lifecycle
 - 5.2 Advertising Interval and Discovery Latency
 - 5.3 BLE 5.0 Extended Advertising
6. SM — Security Management
 - 6.1 Pairing Methods
 - 6.2 The Three Pairing Phases
7. Classic Bluetooth Profiles Overview
8. ESP32-S3 Bluetooth Hardware Features
 - 8.1 Bluetooth and WiFi Coexistence — 2.4 GHz Resource Arbitration
 - 8.2 Coexistence Configuration
9. ESP-IDF Bluetooth Protocol Stack Selection
 - 9.1 Bluedroid vs NimBLE
 - 9.2 Stack Selection
 - 9.3 Key Bluedroid menuconfig Settings
10. Common Problem Troubleshooting

1. Bluetooth Overview

1.1 What Is Bluetooth

Bluetooth is a short-range wireless communication technology operating in the **2.4 GHz ISM band** (2400~2483.5 MHz), used for short-distance data exchange between devices. The name comes from the 10th-century Nordic king Harald Bluetooth (Harald Blåtann), symbolizing the unification of communication between different devices.

Key Parameter	Description
Band	2.4000 ~ 2.4835 GHz (ISM license-free band)
Modulation	GFSK (Gaussian Frequency Shift Keying); BLE 1M / 2M physical layers
Frequency hopping	Adaptive Frequency Hopping (AFH), 1600 times/s (Classic Bluetooth) / pseudo-random hopping (BLE)
Communication range	Typically 10m (Class 2, 0~4 dBm), up to 100m (Class 1, +20 dBm)
Network topology	Point-to-point (Piconet) / Broadcast / Mesh (BLE Mesh)

The name Bluetooth comes from the Danish king Harald "Bluetooth" Gormsson, who unified the tribes of Scandinavia — just as Bluetooth technology aims to unify short-range communication protocols between different devices.

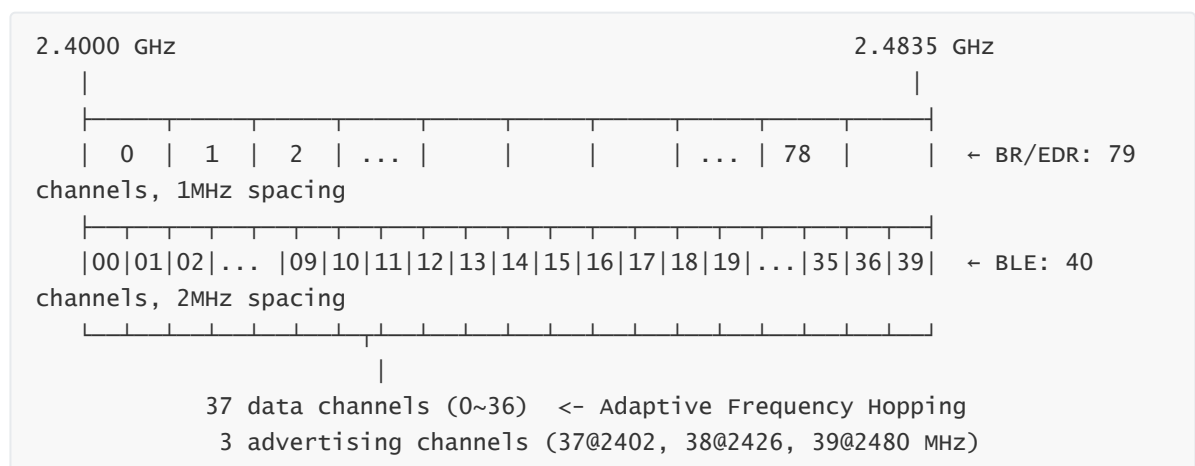
1.2 Bluetooth Version Evolution

Version	Year	Key Features	Max Rate
1.0 / 1.0B	1999	First commercial version, severe interoperability issues	721 kbps
1.1 / 1.2	2001~2003	AFH adaptive frequency hopping to reduce WiFi interference	721 kbps
2.0 + EDR	2004	Enhanced Data Rate (EDR), $\pi/4$ -DQPSK / 8DPSK modulation	2.1 Mbps
2.1 + EDR	2007	Secure Simple Pairing (SSP), reducing pairing complexity	2.1 Mbps
3.0 + HS	2009	Introduced 802.11 (WiFi) as a high-speed channel	24 Mbps
4.0	2010	BLE first introduced , low-power mode; coin-cell batteries last months	1 Mbps (BLE)
4.1 / 4.2	2013~2014	BLE enhancements: LE Secure Connections, data length extension	1 Mbps
5.0	2016	2M PHY, LE Long Range (Coded PHY), advertising extensions	2 Mbps
5.1	2019	AoA / AoD direction finding (centimeter-level)	2 Mbps
5.2	2020	LE Audio, LC3 codec, Isochronous Channels	2 Mbps
5.3 / 5.4	2021~2023	Enhanced channel classification, periodic advertising enhancements, PAwR	2 Mbps

The ESP32-S3 supports **BLE 5.0** and **Classic Bluetooth (BR/EDR)**, with a theoretical maximum application-layer throughput of ~1400 kbps under BLE 2M PHY.

1.3 Bluetooth Band and Channels

The 2.4 GHz ISM band is divided into different channel grids depending on the Bluetooth type:



Bluetooth Type	Number of Channels	Channel Spacing	Hopping Rate
BR/EDR (Classic Bluetooth)	79	1 MHz	1600 hop/s
BLE (Bluetooth Low Energy)	40 (37 data + 3 advertising)	2 MHz	Adaptive pseudo-random

BLE deliberately places its advertising channels at 2402 / 2426 / 2480 MHz — these three frequencies avoid WiFi's most commonly used channels 1, 6, and 11 (2412 / 2437 / 2462 MHz), reducing mutual interference with WiFi at the physical layer.

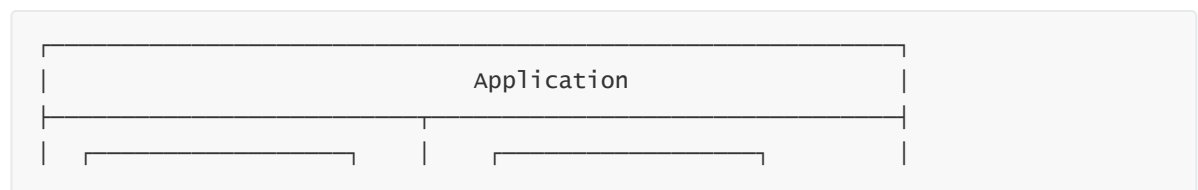
1.4 Bluetooth vs WiFi

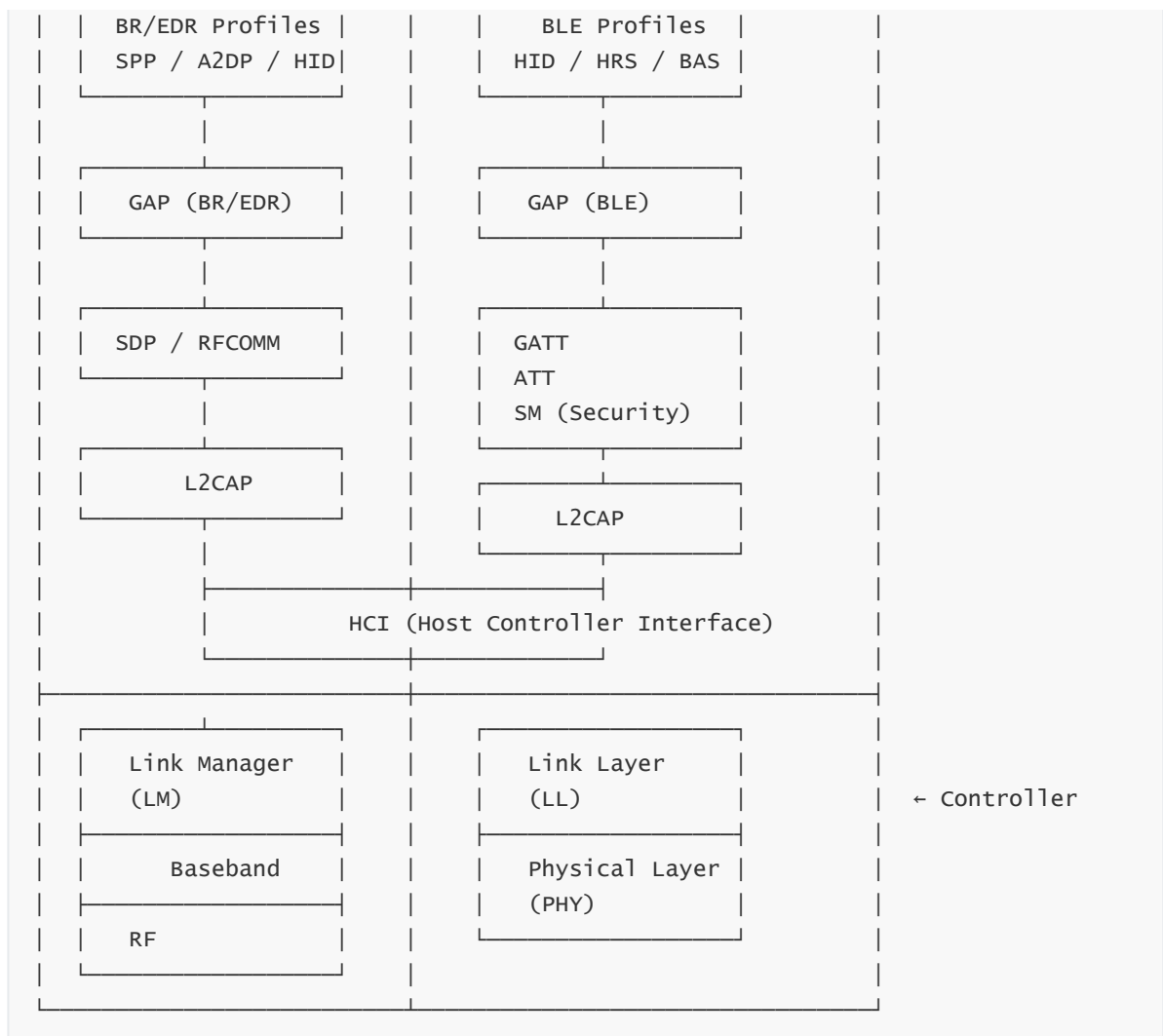
Comparison	Bluetooth (BLE 5.0)	WiFi (802.11n)
Band	2.4 GHz	2.4 GHz / 5 GHz
Channel bandwidth	2 MHz	20 / 40 MHz
Max application-layer rate	~1.4 Mbps	~50 Mbps (ESP32-S3)
Power consumption	Extremely low (μ A-level sleep)	Higher (mA-level continuous reception)
Connection setup	3 ms	100 ms ~ seconds
Typical range	10~100m	30~100m
Network topology	Point-to-point / broadcast / Mesh	Star (AP + STA)
Suitable scenarios	Wearables, sensors, audio	Internet access, video streaming, OTA
IP protocol stack	None (needs upper-layer encapsulation such as 6LoWPAN)	Native TCP/IP

2. Bluetooth Protocol Stack Architecture

2.1 Dual-Mode Architecture

The ESP32-S3 supports **dual-mode Bluetooth** — the same chip can run Classic Bluetooth (BR/EDR) and Bluetooth Low Energy (BLE) simultaneously; both share the RF front end but have independent protocol stacks:





2.2 Host and Controller Layering

The Bluetooth protocol stack is divided into two main parts, communicating through **HCI (Host Controller Interface)**:

Layer	Sub-modules	Responsibilities
Host layer	GAP / GATT / ATT / SM / L2CAP / SDP / RFCOMM	Protocol logic, security management, service discovery
HCI	UART / USB / SDIO transport	Standard communication interface between Host and Controller
Controller layer	Link Layer / Baseband / PHY / RF	RF control, baseband processing, frequency hopping, physical layer modem

On the ESP32-S3, the Host and Controller run on the same chip, and HCI communicates through an internal virtual interface (not a physical UART). On phones and other devices, the Host usually runs on the main CPU and the Controller is a separate Bluetooth chip.

2.3 Detailed Layer Descriptions

Layer	Full Name	Function	Key Concepts
PHY	Physical Layer	Radio transceiver, GFSK modulation	1M / 2M / Coded PHY
LL	Link Layer	Advertising, scanning, connection setup/maintenance, hopping	Advertising state / scanning state / connection state
HCI	Host Controller Interface	Host <-> Controller command/event channel	CMD / Event / ACL Data
L2CAP	Logical Link Control & Adaptation Protocol	Data fragmentation and reassembly, protocol multiplexing	CID (Channel ID), MTU
SM	Security Manager	Pairing, bonding, encryption key distribution	Just Works / Passkey / OOB
ATT	Attribute Protocol	Read/write/notify protocol for attribute data	Attribute Handle / UUID / Permissions
GATT	Generic Attribute Profile	Data organization model	Service -> Characteristic -> Descriptor
GAP	Generic Access Profile	Device discovery and connection management	Advertising / scanning / connection / role definition

3. Classic Bluetooth vs Bluetooth Low Energy

3.1 Core Differences

Feature	Classic Bluetooth (BR/EDR)	Bluetooth Low Energy (BLE)
Physical channels	79 x 1MHz	40 x 2MHz
Hopping rate	1600 hop/s	Adaptive (channel can change at each connection event)
Connection setup	Scan + paging (Page), ~100ms	Advertising -> scan -> connect, ~3ms
Max application-layer throughput	2.1 Mbps	~1.4 Mbps (2M PHY)
Power consumption	Medium (~1W active)	Extremely low (~0.01~0.5W, coin cell lasts months)
Connections	Up to 7 slaves/piconet	No hard limit (determined by stack implementation)
Data model	Serial emulation (SPP/RFCOMM)	Attribute Protocol (ATT/GATT)
Typical applications	Audio streaming, file transfer, Bluetooth headsets	Wearables, sensors, beacons, HID peripherals
Advertising data	Limited (inquiry response)	Rich (max 31 bytes legacy / 1650 bytes extended advertising in BLE 5.0)

3.2 Why BLE Has Low Power Consumption

BLE's low power consumption comes from three design choices:

Design Choice	Classic Bluetooth	BLE	Power Impact
Connection duty cycle	Continuous polling	Connection interval as short as 7.5ms, up to 4s	99% reduction in RF active time
Packet length	Up to 2745 bits	Legacy 376 bits / extended 2120 bits	Shorter transmit duration
Connection latency	None	Can skip N connection events (Slave Latency)	Fewer slave responses
Advertising	Continuous advertising	Configurable interval 20ms ~ 10.24s	Lower advertising RF duty cycle

Classic Bluetooth:  (RF works continuously)

BLE:  (periodic momentary work)

^-----^ Connection Interval (7.5ms~4s)

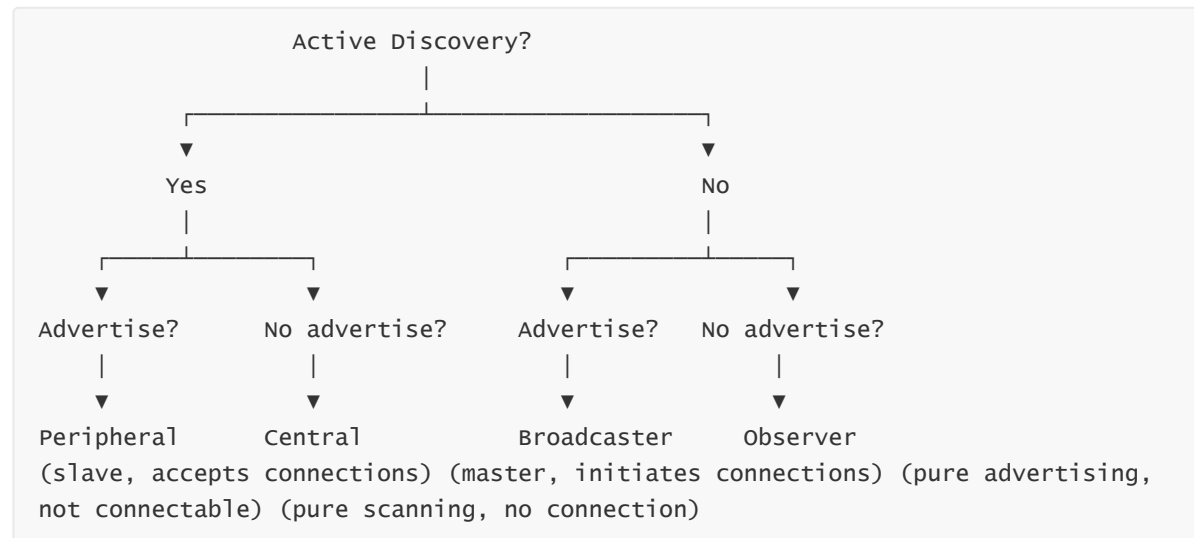
BLE's RF is in **sleep state** most of the time, waking only for a few milliseconds at scheduled connection events to send/receive data. This is the fundamental reason its power consumption is only 1%~5% of Classic Bluetooth's.

4. BLE Core Concepts

4.1 GAP — Generic Access Profile

GAP defines **how BLE devices are discovered and connected by other devices**, and is the entry point for all BLE communication.

The Four GAP Device Roles



GAP Role	English	Sends Advertising	Initiates Connection	Typical Device
Peripheral	Slave/peripheral	☒	✗	Sensors, wristbands, beacons
Central	Master/central	✗	☒	Phones, tablets, gateways
Broadcaster	Pure broadcaster	☒	✗	iBeacon, Eddystone beacons
Observer	Pure observer	✗	✗	Passive listener, packet sniffer

Advertising Types

Advertising PDU Type	Meaning	Connectable	Scan Response Available
ADV_IND	General advertising indication	☒ Connectable	☒ Scan response available
ADV_DIRECT_IND	Directed connection advertising (specified target address)	☒ Connectable	✗
ADV_NONCONN_IND	Non-connectable advertising (pure beacon)	✗	✗
ADV_SCAN_IND	Scannable advertising (not connectable but can obtain more data)	✗	☒

Advertising Data Format

Advertising data is organized in a **Length-Type-Value (LTV)** structure:

```
Advertising packet (max 31 bytes / legacy mode)
|- [L=2] [T=Flags]           [V=0x06]           <- 3 bytes
|- [L=9] [T=Complete Name]   [V="ESP-IDF"]       <- 11 bytes
|- [L=3] [T=16-bit Service]   [V=0xFFFF0]       <- 5 bytes
|- [L=...] other fields...
```

AD Type	Value	Description
0x01	Flags	Discoverability mode + BR/EDR support flags
0x02 / 0x03	Incomplete / Complete 16-bit Service UUIDs	Service UUID list
0x08 / 0x09	Shortened / Complete Local Name	Device name
0x0A	TX Power Level	Transmit power (dBm), used for distance estimation
0xFF	Manufacturer Specific Data	Vendor-specific data (used by iBeacon)

4.2 ATT — Attribute Protocol

ATT (Attribute Protocol) is the fundamental protocol for BLE data transmission. All data on a BLE device is stored in the form of **attributes**; each attribute is a data unit with a handle and a UUID.

Attribute Structure

Attribute Handle	UUID	Permissions	Value	
0x0010	0x2800	R	0xFFFF0	<- Service declaration
0x0011	0x2803	R	NOTIFY, 0xFFFF1	<- Characteristic declaration
0x0012	0xFFFF1	R N	"Hello"	<- Characteristic value
0x0013	0x2902	R W	0x0001	<- CCCD descriptor

Each attribute is defined by four elements:

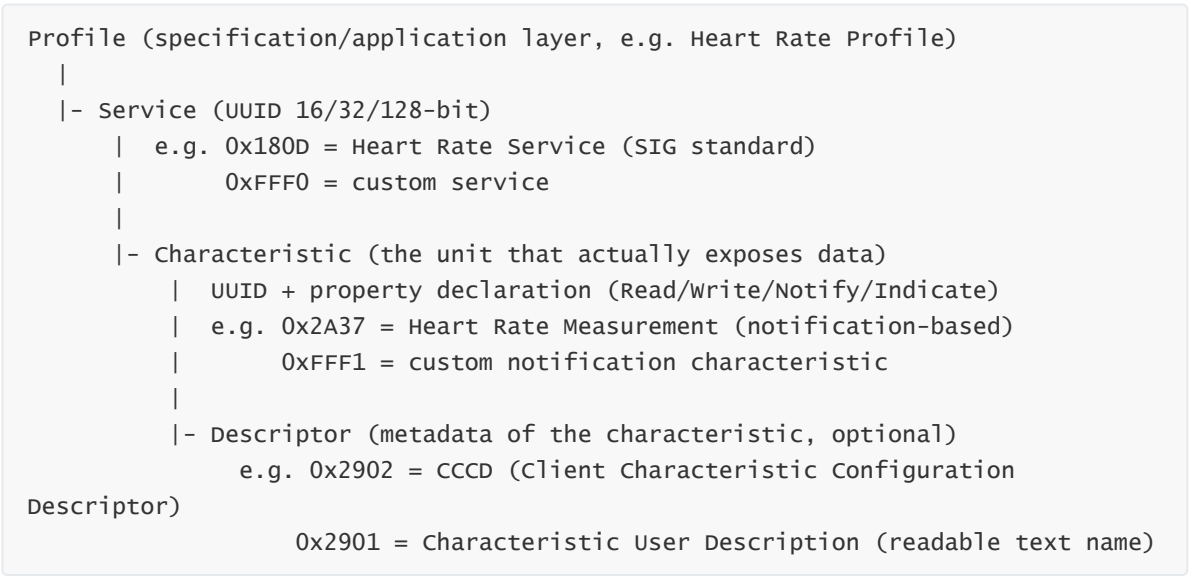
Element	Description	Example
Attribute Handle	16-bit unique number, assigned by the stack	0x0010
Attribute Type (UUID)	Type identifier of the attribute	0x2800 = GATT Primary Service declaration
Permissions	Access permissions (read/write/notify/encrypt)	Read + Notify
Value	Attribute value (fixed-length or variable-length)	"Hello World"

GATT Operations and Permission Mapping

Operation (ATT layer)	GATT Method	Direction	Required Declaration
Read Request / Response	read_cb()	Client <- Server	BLE_GATT_CHR_F_READ
Write Request / Response	write_cb()	Client -> Server	BLE_GATT_CHR_F_WRITE
Write Command (no response)	write_cb()	Client -> Server	BLE_GATT_CHR_F_WRITE_NO_RSP
Handle Value Notification	notify()	Server -> Client	BLE_GATT_CHR_F_NOTIFY
Handle Value Indication	indicate()	Server -> Client (requires confirmation)	BLE_GATT_CHR_F_INDICATE

4.3 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) is built on top of ATT and defines the **hierarchical organization model** of BLE data:



UUID Structure

UUID Type	Length	Format	Example
16-bit UUID (SIG standard)	2 bytes	0xxxxx	0x180D (Heart Rate Ser
32-bit UUID	4 bytes	0xxxxxxxxx	0x12345678
128-bit UUID (custom)	16 bytes	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx	4fafc201-1fb5-459e-8f

16-bit UUIDs are standardized identifiers allocated by the Bluetooth SIG and require a purchased license. Custom projects can freely use 128-bit UUIDs, or use non-reserved 16-bit UUIDs (such as 0xFFFF0~0xFFFFE, which are for development testing only).

CCCD — Client Characteristic Configuration Descriptor

CCCD (Client Characteristic Configuration Descriptor), UUID = 0x2902, is the most critical descriptor in GATT; it controls the on/off switch for the slave's notification/indication:

Central enables notifications: Central -> CCCD (0x2902) -> write 0x0001 -> slave starts pushing Notify

Central disables notifications: Central -> CCCD (0x2902) -> write 0x0000 -> slave stops pushing

+-----+		
CCCD value	Meaning	
+-----+		
0x0000	All notifications/indications disabled	
0x0001	Enable Notify	
0x0002	Enable Indicate	
+-----+		

4.4 Notify vs Indicate Comparison

Comparison	Notify	Indicate
Waits for confirmation after sending	✗ No	☑ Waits for application-layer ACK
Send rate	Fast (no confirmation wait)	Slow (waits for ACK each time)
Reliability	Low (no retransmission on loss, except link-layer retransmission)	High (application-layer confirmation + retransmission)
Typical scenario	High-frequency sensor reporting (heart rate, temperature)	Critical commands (alarms, on/off control)

Notify flow:

slave --data--> central
(no confirmation needed,
continue sending next)

Indicate flow:

slave --data--> central
<--ACK--
slave sends the next one only after receiving ACK

4.5 BLE Connection Parameters

After a BLE connection is established, the central can request to modify the following connection parameters from the peripheral:

5.2 Advertising Interval and Discovery Latency

The smaller the advertising interval, the faster the device is discovered, but the higher the power consumption:

Advertising Interval	Average Discovery Latency	Average Power	Suitable Scenario
20ms (fast advertising)	~10ms	~1.5mA	Initial provisioning stage
100ms	~50ms	~0.4mA	Normal connectable state
1s (slow advertising)	~500ms	~0.05mA	Long-term standby, beacons

Android / iOS have **scan window limitations** — iOS scans in the background about once every 1~2s, so the advertising interval should not exceed 500ms, otherwise the phone may take a long time to discover the device. Some systems also require the advertising data to contain a specific Service UUID before the App can filter it in the background.

5.3 BLE 5.0 Extended Advertising

BLE 5.0 introduced the extended advertising mechanism, greatly increasing the advertising data capacity from 31 bytes:

Legacy advertising (BLE 4.x):

3 advertising channels, each packet $\leq$ 31 bytes payload

Extended advertising (BLE 5.0):

Supports carrying advertising data on 37 data channels (not on advertising channels),

can send multiple packets in a chain (max 1650 bytes payload)

Applications: indoor navigation positioning, asset tracking, batch e-paper price tag updates

6. SM — Security Management

6.1 Pairing Methods

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3. Thonny IDE`.

BLE Security Management (SM) is responsible for **pairing** and **bonding** between devices:

Pairing Method	Security	User Interaction	Suitable Scenario
Just Works	Low (no MITM protection)	No interaction	Sensors without UI (temperature/humidity meters)
Passkey Entry	Medium	One side displays a 6-digit number, the other enters it	Keyboard to phone, phone to device with screen
Numeric Comparison	High (LE Secure Connections)	Both sides display a 6-digit number, user confirms they match	Recommended for BLE 4.2+, phone to peripheral
Out of Band (OOB)	Highest	Keys exchanged through an external channel such as NFC / QR code	Enterprise security, access control systems

6.2 The Three Pairing Phases

Phase 1: Feature Exchange

Both sides exchange IO Capabilities (display/keyboard availability) and security requirements

-> negotiate the pairing method (Just Works / Passkey / OOB)

Phase 2: Key Generation

STK (Short Term Key, legacy pairing, BLE 4.0/4.1)

or LTK (Long Term Key, LE Secure Connections, BLE 4.2+)

-> establish an encrypted link

Phase 3: Key Distribution

LTK + IRK (Identity Resolving Key) + CSRK (Connection Signature Resolving Key)

-> bonding complete; subsequent reconnections are automatically encrypted (no re-pairing needed)

Pairing is not bonding. Pairing is a one-time encryption process; bonding stores the keys in NVS flash so future connections are automatically encrypted — this is the "paired devices list" seen in Flutter / Android Apps.

7. Classic Bluetooth Profiles Overview

Although BLE is the IoT mainstream, the ESP32-S3 also supports Classic Bluetooth, which can be used for the following classic Profiles:

Profile	Full Name	Typical Application	ESP32-S3 Support
SPP	Serial Port Profile	Wireless serial transparent transmission — emulates UART to replace wired serial	☒ Most commonly used
A2DP	Advanced Audio Distribution Profile	Bluetooth audio streaming (headsets, speakers)	☒
AVRCP	Audio/Video Remote Control Profile	Bluetooth remote control (previous/next track, volume)	☒
HID	Human Interface Device	Bluetooth keyboard/mouse/gamepad	☒
HFP	Hands-Free Profile	Car hands-free calling	☒

SPP is the most commonly used profile in ESP32-S3 Classic Bluetooth development — it establishes a virtual serial port between two devices through the RFCOMM serial emulation protocol. To upper-layer code it appears as sending/receiving byte streams, similar to UART but without physical wiring. Suitable for wireless debugging, transparent control, data collection, and other scenarios.

8. ESP32-S3 Bluetooth Hardware Features

Feature	Parameter
Bluetooth standard	BLE 5.0 + BR/EDR (Classic)
Operating band	2.4 GHz ISM
Max transmit power	+20 dBm (Class 1)
Max receive sensitivity	-97 dBm (BLE 1M)
BLE max throughput	~1.4 Mbps (2M PHY, application layer ~700 kbps)
Classic Bluetooth max rate	2.1 Mbps (EDR)
Antenna	On-board PCB ceramic antenna / IPEX external antenna
Protocol stack	Bluedroid (full) / NimBLE (BLE only)

8.1 Bluetooth and WiFi Coexistence — 2.4 GHz Resource Arbitration

Bluetooth and WiFi share the same antenna and 2.4 GHz RF front end — **the two cannot transmit simultaneously**. The ESP32-S3 resolves the conflict through **TDM (Time Division Multiplexing)**:

Comparison	Bluedroid	NimBLE
Bluetooth types	Dual-mode (BR/EDR + BLE)	BLE only
ROM usage	~512 KB	~32 KB
RAM usage	~120 KB (minimum)	~4.5 KB
Feature completeness	Full: SPP, A2DP, HID, full-featured GATT	Streamlined: GAP + GATT only, main BLE 5.0 features
Configuration complexity	High, many menuconfig options	Low, ~10 lines of code to initialize
Suitable scenarios	Need Classic Bluetooth (audio, SPP transparent) or full BLE features	BLE-only, code-size-sensitive scenarios
ESP-IDF default	☒ Enabled by default	Must be selected manually

9.2 Stack Selection

```

Do you need Classic Bluetooth (SPP/A2DP/HID)? --yes--> Bluedroid (only choice)
|
no (BLE only)
|
|-- need full BLE features (Mesh, Extended Advertising, LE Audio)?
|     --yes--> Bluedroid
|
|-- streamlined BLE peripheral/central roles, code/memory sensitive?
|     --yes--> NimBLE

```

The BLE peripheral and central experiments later in this project both use the **NimBLE** stack. For IoT teaching scenarios that only need BLE peripheral/central roles, NimBLE's code size and memory overhead are far smaller than Bluedroid's, making it the more suitable choice.

9.3 Key Bluedroid menuconfig Settings

When using Bluedroid, you must configure:

```

Component config -> Bluetooth
|- [*] Bluetooth
|- Bluetooth Host -> Bluedroid
|- Bluedroid Options
|   |- [*] Include GATT
|   |- [ ] Include Classic Bluetooth (as needed)
|   |- [*] Enable BLE 5.0 features (as needed)

```

When using NimBLE:

```

Component config -> Bluetooth
|- [*] Bluetooth
|- Bluetooth Host -> NimBLE
|- Bluetooth Controller -> BLE Only

```

10. Common Problem Troubleshooting

Symptom	Cause	Solution
Crash after Bluetooth initialization	NVS not initialized or partition corrupted	Add try-erase logic to <code>nvs_flash_init()</code> ; or erase the entire flash from the IDE
Advertising fails to start with <code>rc=12</code>	BLE controller not enabled	In menuconfig, ensure <code>CONFIG_BT_ENABLED=y</code> and controller mode = BLE Only
Phone cannot scan the ESP32-S3	Advertising interval too large / no Service UUID	Lower the advertising interval to $\leq 100\text{ms}$; confirm the advertising data contains Flags + Name
Disconnects immediately after connecting (0~2s)	Incompatible connection parameters / too far away	Get closer to the device ($\leq 5\text{m}$); confirm no strong same-band WiFi interference
WiFi disconnects frequently during BLE connection	2.4G RF conflict	Set the coexistence policy to <code>ESP_COEX_PREFER_WIFI</code> ; or increase the BLE connection interval to $\geq 100\text{ms}$
iOS cannot scan the device in the background	iOS does not notify Apps of advertising without a Service UUID	Add a specific Service UUID to the advertising data (the iOS App must declare the corresponding background mode)
GATT cannot discover the custom service	Wrong NimBLE service table definition	Confirm the <code>ble_gatt_svc_def</code> array is terminated with <code>{0,}</code> and the UUID format is correct
Notify cannot be sent	CCCD not enabled / invalid conn_handle	Confirm the central has written CCCD = 0x0001; confirm conn_handle matches the current connection
<code>ATT write not permitted</code> log	Wrote to a characteristic value handle without Write permission	Confirm the write targets the CCCD descriptor handle rather than the characteristic value handle; or declare the Write flag for the characteristic
Bluetooth interrupted during OTA	Underlying restart during OTA resets the Bluetooth driver	After OTA, re-run <code>nvs_flash_init()</code> + <code>nimble_port_init()</code>

